Here is a complete, step-by-step implementation plan and timeline to build your T20 Cricket outcome prediction platform from scratch, hitting all 10 points of your hackathon roadmap while architecting it as a monetizable, production-ready SaaS tool.

Phase 1: Data Acquisition & Live Simulation (Weeks 1-2)

- **Step 1: Data Acquisition:** Start by downloading historical ball-by-ball JSON or CSV data from sources like Cricsheet or Kaggle, which will give you the necessary metadata, match info, and innings data.
- **Step 2: Real-Time Simulation:** To simulate live matches for the hackathon, build a Python producer script that reads your static data and pushes it into an Apache Kafka stream. Use a generator pattern for memory efficiency and inject randomized temporal delays between deliveries to accurately mimic the real-time cadence of a live T20 match.

Phase 2: Data Engineering & Quality Governance (Weeks 3-4)

- **Step 3: Medallion Architecture & Data Warehouse:** Set up your cloud data warehouse (e.g., PostgreSQL or Snowflake) using a Medallion Architecture.
    - *Bronze Layer:* Ingest the raw Kafka event streams to maintain an immutable historical archive.
    - *Silver Layer:* Cleanse, deduplicate, and standardize the data.
    - *Gold Layer:* Build a dimensional Star Schema containing fact tables (e.g., deliveries, match summaries) and dimension tables (players, venues, dates).
- **Step 4: ETL Pipeline:** Use dbt (data build tool) to orchestrate these transformations. Apply rigorous data quality rules here, such as validating null values and establishing statistical boundaries for outliers.
- **Step 5: EDA & Quality Dashboard:** Build an internal dashboard using Streamlit or Dash. Connect it directly to your dbt outputs to visualize pipeline health, quarantine tables, and historical trends.

Phase 3: Machine Learning & Analytics (Weeks 5-6)

- **Step 6: Persona-Based Dashboards:** Identify your target audiences and their KPIs.
    - *Coaches:* Phase-specific strike rates and player matchups.
    - *Bettors/Scouts:* Dynamic win probabilities and situational pressure indexes.

    - Build these tailored UI views using a modern frontend framework like React or Next.js.

- **Step 7: ML Use Cases:** Implement an XGBoost or Random Forest classification model to predict the match outcome. Your feature engineering is critical here; calculate dynamic features like balls remaining, wickets in hand, required run rates, and rolling team form.

Phase 4: GenAI & Inference Optimization (Weeks 7-8)

- **Step 8: GenAI Integration:** Build an Agentic RAG system using LangGraph and LangChain. Because sports data is heavily numerical, standard vector RAG won't work well. Implement a Text-to-SQL tool where the LLM is given your database schema and writes SQL queries dynamically to answer complex user questions (e.g., "What is the win probability if chasing 180 at Eden Gardens?").
- **Step 9: ML Optimization:** To ensure your SaaS can deliver sub-second updates to live bettors, optimize your inference pipeline. Use tools like Ray Serve, which handles dynamic request batching and horizontal scaling for online inference APIs.

Phase 5: Multi-Tenant Deployment & Monetization (Weeks 9-10)

- **Step 10: Dockerized Deployment:** Containerize your entire stack (Next.js frontend, FastAPI backend, Kafka, PostgreSQL, and ML microservices) using Docker Compose for simple orchestration and horizontal scaling.
- **SaaS Architecture:** To support multiple clients securely, implement Row-Level Security (RLS) in your database. This tags every row with a tenant_id, ensuring that Franchise A can never see Franchise B's proprietary data despite sharing the same database infrastructure.
- **Monetization (Stripe):** Integrate Stripe via webhooks into your FastAPI backend to manage subscription tiers. You can offer a "Freemium" tier for casual fans, a "Pro" tier for bettors ($15/mo), and an "Enterprise" tier for teams requiring API access.
- **Production Monitoring:** Finally, deploy Prometheus and Grafana alongside your stack. Use these to monitor system health, tenant-specific API usage, and critical ML model metrics like prediction drift (when real-world behaviors start shifting away from your training data) to know exactly when to retrain your XGBoost models,.